Digital Images & the Pipeline

Week 1 — Session 2: Images in Code & the Imaging Model

Practical Image Processing

Lecture 1.2 ▪ 60–70 min

Recap of Session 1

Last session we established what an image *is*:

- A function $f(x, y)$ **sampled** onto a grid and **quantized** to integer levels.
- Resolution and bit depth set file size and detail.
- Color = three numbers per pixel; luminance $\neq$ naive average.
- HSV separates color from brightness for robust selection.

Today

Those numbers, *in code*: NumPy arrays, the `uint8` trap, file formats, and the physical model that creates every pixel.

The pipeline, again — today's slice



Key idea

Session 2 lives at **Acquire**: how pixels enter your program, how they are stored, and where their imperfections come from.

Roadmap

Images in code

In-class exercise

Formats & storage

Deep dive: the imaging model

Wrap-up

Images in code

An image is a NumPy array

```python
import cv2, numpy as np
img = cv2.imread('page.jpg')          # BGR, shape (H, W, 3), dtype uint8
print(img.shape, img.dtype)           # e.g. (1200, 900, 3) uint8
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # (H, W)
print(gray[0, 0])                     # a single pixel's intensity, 0..255
```

✕ OpenCV gotcha

OpenCV loads images as **BGR**, not RGB. Forgetting this flips red and blue everywhere.

Shape and dtype — read them first

```
print(img.shape)      # (1200, 900, 3)  -> H=1200, W=900, 3 channels
print(img.ndim)       # 3  (color); a grayscale image has ndim == 2
print(img.dtype)      # uint8  -> values 0..255, wraps on overflow
print(img.min(), img.max(), img.mean())   # exposure at a glance
```

- `shape[0]` is height (rows), `shape[1]` is width (columns) — rows come first.
- `dtype` decides the arithmetic. `uint8` is fast and compact but *wraps*.

Note

A low `mean` = a dark image; a `max` well below 255 = never using full brightness.

Pixels, slices, and vectorized ops

```python
h, w = gray.shape
roi = gray[100:200, 50:150]           # a rectangular region (a view)
gray[gray < 50] = 0                    # vectorized: threshold dark pixels to black
brighter = np.clip(gray.astype(int) + 40, 0, 255).astype(np.uint8)
```

✔ Good practice

Never loop over pixels in Python — it's 100× slower. Use NumPy vectorization / OpenCV.

Slicing is $[y, x]$, and a slice is a view

```
crop = meter[10:38, 60:120]   # rows 10..37, cols 60..119  -> [y, x]!
crop[:] = 0                   # writes THROUGH to `meter` (a view, not a copy)
safe = meter[10:38, 60:120].copy()   # take a copy when you need independence
```

- Rows (y) before columns (x) — the opposite order of a math point (x, y).
- A basic slice shares memory. Mutating it edits the original unless you `.copy()`.

✕ Pitfall

"My original changed by itself" almost always means you mutated a view.

Show and save

```python
import matplotlib.pyplot as plt
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.axis('off'); plt.show()
cv2.imwrite('gray.png', gray)          # PNG = lossless; JPG would re-compress
```

Note

Convert BGR→RGB before `plt.imshow`, or your colors look wrong.

The uint8 overflow trap — seen on real data



+60 done right (clip): uniformly brighter



+60 in uint8: bright pixels *wrapped* to dark

✖ Pitfall

$200+100 = 44$, not 255. Cast to `int`, compute, `clip` to 0–255 — or use `cv2.add` (saturating). A "brighten" that darkens bright areas is this bug.

Why $200 + 100 = 44$

The arithmetic behind the trap

`uint8` holds 0–255: eight bits, 256 states. Addition is *modulo* 256.

$200 + 100 = 300$, and $300 \bmod 256 = 300 - 256 = 44$.

The value "wraps" past 255 back to 0 and keeps counting — bright turns dark.

- `np.clip(x.astype(int)+100, 0, 255)` promotes to a wide type, then clamps: 255.
- `cv2.add(x, 100)` **saturates** in place: anything over 255 sticks at 255.
- Same fix, two styles. Both refuse to wrap.

In-class exercise

In-class exercise (10 min)

Predict, then run

Write your answers down *before* executing the cell.

```
gray = np.array([[100, 150],[200, 250]], dtype=np.uint8)
a = gray + 60          # (1) what are the four values?
b = cv2.add(gray, 60)  # (2) and here?
c = gray.mean()        # (3) what does this represent?
```

1. Compute `a` by hand — watch the overflow.
2. What is `c`, and what would a very low `c` tell you about the image?

2. How does `b` differ?

Debrief

- `a` = `[[160,210],[4,54]]` — 200+60 and 250+60 *wrapped* (overflow).
- `b` = `[[160,210],[255,255]]` — `cv2.add` saturates (clamps to 255).
- `c` = mean intensity (≈175); a very low mean means a mostly dark image (under-exposed).

Key idea

The mean/histogram of an image tells you about exposure before you look at it — Week 2.

Formats & storage

Image formats: lossless vs lossy

Format	Compression	Use
PNG	lossless	masks, line art, intermediate results
JPEG	lossy	photos; small files, but artifacts
TIFF	lossless/none	archival, scientific, multi-page
BMP/RAW	none	simplicity / sensor data

Key idea

Never save intermediate results as JPEG — its artifacts corrupt later processing. Use PNG until the very end.

JPEG artifacts matter for processing

- JPEG discards high-frequency detail in 8×8 blocks — invisible to you, poison to edge detection.
- Re-saving a JPEG repeatedly compounds the damage (generation loss).
- For DocScan, work from the least-compressed source you can get.

Note

A crisp digit edge is exactly the high-frequency content JPEG throws away first — the meter and plate crops suffer most where we most need sharpness.

Compression trade-off, in numbers

Worked example

A 4000×3000 color photo is 36 MB uncompressed. Typical stored sizes:

format	size	ratio
raw / BMP	36 MB	1:1
PNG (lossless)	∼12–20 MB	∼2–3:1
JPEG q=90	∼3–5 MB	∼8–12:1
JPEG q=50	∼1 MB	∼36:1

- Lossless buys modest savings for free; lossy buys big savings by *discarding* data.
- The savings you cannot see are still gone — decide before, not after, processing.

Bit depth & dynamic range

- 8-bit is fine for display; 12–16-bit sensors keep detail in shadows/highlights.
- **Dynamic range** = ratio of brightest to darkest a sensor can capture.
- HDR combines exposures to exceed a single sensor's range (beyond our scope, good to know).

Note

Dynamic range is often quoted in **stops** (powers of 2): a 12-stop sensor spans a $2^{12} = 4096{:}1$ range from darkest to brightest.

Deep dive: the imaging model

Sampling theory & aliasing

- **Nyquist**: to capture detail at frequency f, sample at $> 2f$. Under-sample and high frequencies masquerade as low ones.
- That's **aliasing**: moiré patterns, jagged edges, wagon-wheels spinning backwards.
- Fix: blur (low-pass) *before* down-sampling — remove detail you can't represent (Week 3, 6).

✕ Pitfall

Resizing an image down without blurring first creates aliasing artifacts that no later step can undo.

Aliasing, made concrete

Worked example

Suppose the digits on the meter repeat fine texture at $f = 0.4$ cycles/pixel. Nyquist demands a sampling rate above $2f = 0.8$.

Down-sample by 2 (keep every other pixel): the new rate is 0.5 — *below* 0.8. That 0.4 texture folds to an alias at $|0.5 - 0.4| = 0.1$ cycles/pixel: a phantom low-frequency ripple.

- The fake ripple sits right in the signal band — no later filter can separate it out.
- Blur first (kill everything above 0.25), *then* down-sample: nothing is left to fold.

Gamma: pixels aren't linear light

- Displays and files are **gamma-encoded**: pixel value $\neq$ linear light intensity.
- Storing perceptually (more codes for dark tones, where eyes are sensitive) saves bits.
- For *correct* math (averaging, blending), sometimes linearize first — a common subtle bug.

Math

Encoded $\approx$ (linear)$^{1/\gamma}$, with $\gamma \approx 2.2$. We'll use gamma for contrast in Week 2.

Gamma, made concrete

Worked example

Half the linear light is *not* pixel value 128. Encoding 0.5 with $\gamma = 2.2$:

$0.5^{1/2.2} = 0.5^{0.4545} \approx 0.73$, so the stored 8-bit code is $0.73 \times 255 \approx 186$, not 128.

Conversely, code $128/255 = 0.502$ decodes to $0.502^{2.2} \approx 0.22$ of full light.

✕ Pitfall

Averaging two gamma-encoded pixels is not the average of their light. For physically correct blending, linearize $\rightarrow$ average $\rightarrow$ re-encode.

Where pixels come from



Real sensors capture one color per pixel (a **Bayer** mosaic) and interpolate the rest. Noise, blur, and color casts enter here — which is exactly what we clean up (Weeks 4, 2).

Noise is physical, not a bug

- Photon counting is random $\rightarrow$ shot noise; electronics add read noise.
- Low light / high ISO = more noise. It's the raw material Week 4 removes.
- Understanding *where* noise comes from tells you *how* to model and remove it.

Note

Shot noise follows a Poisson law: its standard deviation grows as $\sqrt{N}$ with the photon count N. Brighter pixels are noisier in absolute terms but *cleaner* in relative terms.

Coordinates & conventions

- Images are indexed `[row, col]` = `[y, x]` — the *opposite* of math (x, y).
- Origin is top-left; y increases *downward*. This trips up everyone once.
- OpenCV points are `(x, y)`; NumPy indices are `[y, x]`. Keep them straight.

✕ Pitfall

Swapping x and y is the #1 beginner bug in image code. When something's transposed, check this first.

Wrap-up

What you should now be able to do

- Load an image, inspect its shape/dtype, convert color, and slice regions — vectorized.
- Explain and avoid the `uint8` overflow, BGR-vs-RGB, view-vs-copy, and x/y traps.
- Choose a format for a task and predict its compression trade-off.
- Name where noise, blur, aliasing, gamma, and color casts physically come from.

This week's deliverable

Due before Week 2 (see `project/`)

1. Load 3 images; print shape, dtype, min/max/mean; convert to grayscale and to HSV.
2. Write a correct `brighten(img, k)` (no overflow) and compare to `cv2.add`.
3. Take a phone photo of a document — your DocScan test image. Report its properties.
4. Short note: for your photo, which stages of the pipeline will be hardest, and why?

✓ Good practice

Save every intermediate result as PNG, and print `shape`/`dtype` at each step — the two habits that prevent most Week 1 bugs.

An image is just numbers on a grid.

Everything else in this course is arithmetic on those numbers.